Impression 3D TP1

Se préparer pour le développement sous Eclipse

Ouvrez Eclipse JDT (Java Development Tools) et utilisez le code fourni. Ce code comporte :

- Un embryon d'application (fc.PrintingApplication.TP1)
- Des dépendances (bibliothèques). Il s'agit, notamment, d'une bibliothèque permettant de lire des objets 3D au format OBJ (com.owens.oobjloader.builder), et une autre contenant des outils mathématiques (fc.Math). Utilisez les classes de la bibliothèque mathématique chaque fois que vous le pouvez.
  Les autres dépendances doivent être importées mais vous n'avez pas à vous en préoccuper (fc.Serialization et fc.ComponentModel).

Décompressez directement l'archive dans votre répertoire Workspace Eclipse. Vous importez les packages fournis dans la fenêtre 'Package Explorer', puis 'Importer…' puis 'Existing projects into Workspace'. A l'ouverture sous Eclipse, les dépendances sont automatiquement résolues.

Attention : ne vous créez **pas** un seul package Eclipse contenant tout le code. Laissez les packages importés vivre individuellement les uns des autres et avoir des dépendances entre eux.

Le code ne comporte pas de fonction main(), c'est à vous de l'ajouter. Une fonction déjà écrite vous indique comment énumérer les sommets et les faces du fichier OBJ. Les 'faces' sont des polygones. Ces polygones ont généralement 3 côtés (triangles), mais peuvent en avoir davantage avec certains fichiers OBJ. Elles seront à découper en triangles.

Objectif du TP - Extraction fiable de tranche vectorielle

Depuis votre code Java vous devez :

- Calculer la boîte englobante du fichier OBJ. Vos tranches vont se situer dans cette boîte, par pas vertical (vertical = axe Z) de 0,20 mm.
  Remarque : il vous est demandé d'augmenter la taille de la boite englobante d'1 cm sur chaque côté X et Y (pas sur Z). Les coordonnées-monde étant systématiquement exprimées en millimètres, si votre modèle a une boîte englobante de {0,0,0}-{100,100,100} (donc une taille de 100 mm = 10 cm), fabriquez une boite {-10,-10,0}-{110,110,100}. C'est cette boite qui va vous servir d'espace de tranchage. Attention, il n'y a pas de marge sur Z.
- Les modèles fournis sont étanches, i.e. ils ne présentent pas d'erreur topologique, pas de trou. Toutes les tranches que vous pouvez obtenir seront elles-mêmes topologiquement correctes. **L'étanchéité est garantie, et est indispensable !**
  Les tranches obtenues doivent être des segments de ligne en coordonnées 3D (eg. des arêtes). Tous les segments dans une même tranche auront des sommets avec la même coordonnée Z.
- Faites le rendu de vos tranches dans des bitmaps. La résolution d'une bitmap doit être calculée de la manière suivante : 1 pixel doit correspondre à 0,05 mm.

- Une fois la routine de tranchage terminée, écrire une boucle faisant varier sur le plan Z la hauteur de coupe, par pas de 0,20 mm. Sauvegarder les images bitmap obtenues à chaque tranchage et observer les résultats sur disque avec un visualiseur d'image.
- Le résultat de ce TP servira de base pour les autres TP, soignez votre code.

Rappels

On rappelle que pour dessiner des formes, la classe BufferedImage peut être utilisée :

Pendant de ce TP vous tracerez les données dans une image :

```
BufferedImage image = new BufferedImage(<largeur>, <hauteur>,
   BufferedImage.TYPE_INT_RGB);

image.setRGB(int x, int y, int couleur); // trace un point. Format de couleur 0xRRVVBB

// Rechercher sur internet pour tracer des lignes etc. (voir classe Graphics2D)

// Pour sauver l'image finale
try { ImageIO.write(image, "png", new File("resultat.png")); }

catch (Exception e) { e.printStackTrace() ;
```